

DAM · 2º CURSO · NEBRIJA

Repaso de Java

De cero a la conexión con bases de datos — con las interfaces funcionales explicadas de verdad, a fondo y con muchos ejemplos

Fundamentos (repaso rápido)

POO (4 pilares)

equals / hashCode

Genéricos

Interfaces funcionales a fondo

Streams

Excepciones & Ficheros

JDBC & MongoDB

JUnit

Hecho para Jorge — para ir con ventaja en Acceso a Datos

Índice

1. **1.** Introducción a Java y cómo funciona
2. **2.** Tipos de datos, variables y autoboxing
3. **3.** Entrada y salida de datos
4. **4.** Operadores
5. **5.** Estructuras de control (if / switch)
6. **6.** Bucles (for / while / do-while)
7. **7.** Arrays
8. **8.** Métodos (funciones)
9. **9.** Modificadores de acceso y `static` / `final`
10. **10.** Clases, objetos, getters y setters
11. **11.** Los 4 pilares de la POO
12. **12.** `equals()` , `hashCode()` y `toString()`
13. **13.** Interfaces vs. clases abstractas
14. **14.** Enums (enumeraciones)
15. **15.** Records
16. **16.** Genéricos (Generics)
17. **17.** Colecciones (List, Set, Map)
18. **18.** Qué es una interfaz funcional (SAM) y `@FunctionalInterface`
19. **19.** Lambdas a fondo: sintaxis, captura de variables y "this"
20. **20.** `Function<T,R>` a fondo: `andThen` , `compose`
21. **21.** `Predicate<T>` a fondo: `and` , `or` , `negate`
22. **22.** `Consumer<T>` y `Supplier<T>` a fondo
23. **23.** `BiFunction` , `BiPredicate` , `BiConsumer`
24. **24.** `UnaryOperator` y `BinaryOperator`
25. **25.** Especializaciones primitivas: evitar el autoboxing
26. **26.** Otras interfaces funcionales: `Runnable` , `Callable` , `Comparator`
27. **27.** Referencias a métodos a fondo: los 4 tipos
28. **28.** Streams de principio a fin: un pipeline grande con todo
29. **29.** Errores comunes y buenas prácticas con lambdas
30. **30.** Chuleta final: todas las interfaces de `java.util.function`
31. **31.** Excepciones (manejo de errores)
32. **32.** Lectura y escritura de ficheros
33. **33.** Principios de diseño: SRP y programar contra interfaces
34. **34.** Arquitectura en capas (Model / Service / Controller)
35. **35.** Conexión a bases de datos: JDBC (SQL) y MongoDB (NoSQL)
36. **36.** Testing básico con JUnit
37. **37.** Debugging: cómo pensar como programador
38. **38.** JavaDoc y buenas prácticas generales
39. **39.** Resumen y plan de repaso

Parte 1 · Fundamentos del lenguaje (repaso rápido)

Esta parte va rápida a propósito — es lo que ya viste el año pasado, así que la tratamos como calentamiento antes de entrar en lo bueno.

1. Introducción a Java y cómo funciona

Java es un lenguaje **orientado a objetos**, **fuertemente tipado** y **multiplataforma**, gracias al lema "write once, run anywhere": el mismo programa compilado se ejecuta igual en Windows, Linux o macOS, siempre que haya una JVM instalada.

1. **Escritura:** código fuente en un archivo `.java`.
2. **Compilación:** `javac` lo traduce a **bytecode** (`.class`).
3. **Ejecución:** la **JVM** carga ese bytecode y lo ejecuta, con un compilador JIT optimizando en caliente y un **garbage collector** liberando memoria sola.

```
public class Programa {  
    public static void main(String[] args) {  
        System.out.println("Hola mundo");  
    }  
}
```

2. Tipos de datos, variables y autoboxing

- **Primitivos:** `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. Guardan el valor directamente.
- **De referencia:** `String`, arrays, tus clases. La variable guarda una dirección al objeto real.

```
int entero = 10;  
long enteroLargo = 100L;  
double decimalDoble = 3.1416;  
char letra = 'A';  
boolean activo = true;  
String texto = "Hola mundo";  
var lista = new ArrayList<String>(); // var: el compilador infiere el tipo
```

Autoboxing/unboxing: cada primitivo tiene su clase envoltorio (`int` → `Integer` ...), necesaria para meter números en colecciones. Java convierte de uno a otro automáticamente.

Tipo	Tamaño	Rango aproximado
<code>byte</code>	8 bits	-128 a 127
<code>short</code>	16 bits	-32.768 a 32.767
<code>int</code>	32 bits	±2.147 millones aprox.
<code>long</code>	64 bits	±9,2 trillones aprox. (necesita el sufijo <code>L</code>)
<code>float</code>	32 bits	Decimal, precisión limitada (sufijo <code>f</code>)
<code>double</code>	64 bits	Decimal, la opción por defecto para decimales

Por defecto usa `int` para enteros y `double` para decimales — son los tipos que Java asume si no especificas nada, y sobran para el 95% de los casos de un DAM. Solo bajas a `byte` / `short` con arrays enormes donde la memoria importa de verdad.

== vs equals: con objetos, `==` compara si son el mismo objeto en memoria; `equals()` compara el contenido. Con primitivos, `==` compara el valor directamente. Le dedicamos la sección 12 entera a esto.

2.1 Casting entre primitivos: widening y narrowing

```
int entero = 10;  
double comoDouble = entero; // widening: int → double, AUTOMÁTICO, no se pierde información  
  
double decimal = 9.99;  
int comoEntero = (int) decimal; // narrowing: double → int, hace falta cast EXPLÍCITO  
// comoEntero vale 9 — trunca, no redondea
```

Widening (de un tipo "más pequeño" a uno "más grande": `int` → `long` → `float` → `double`) es automático porque nunca se pierde información. **Narrowing** (al revés) exige un cast explícito precisamente porque el compilador te está avisando: "aquí puedes perder datos, confirma que lo sabes".

Para redondear en vez de truncar: `Math.round(decimal)` (devuelve `long`), no un cast a `int`.

3. Entrada y salida de datos

```
Scanner sc = new Scanner(System.in);
System.out.print("Introduce un número: ");
int num = sc.nextInt();
sc.close();
```

`sc.nextInt()` con texto que no es un número lanza `InputMismatchException` (sección 31) — envuélvelo en un `try/catch` si el dato viene de un usuario real.

3.1 Trabajando con `String` : los métodos que se usan de verdad

```
String texto = "  Hola Jorge  ";
texto.trim();           // "Hola Jorge" — quita espacios de los extremos
texto.strip();           // como trim(), pero entiende unicode (mejor en general)
texto.toUpperCase();     // "  HOLA JORGE  "
texto.contains("Jorge"); // true
texto.replace("Jorge", "Ana"); // "  Hola Ana  "
texto.split(" ");        // String[] — divide por el separador (ojo con espacios repetidos)
String.join(", ", "Ana", "Luis"); // "Ana, Luis"
String.format("%s tiene %.1f de nota", "Ana", 8.5); // "Ana tiene 8.5 de nota"
```

`String` es inmutable: ningún método anterior modifica `texto` — todos devuelven un `String` nuevo. Si no guardas el resultado (`texto = texto.trim();`), el cambio se pierde.

```
// ✗ Mal para muchas concatenaciones en un bucle: crea un String nuevo en CADA vuelta
String resultado = "";
for (int i = 0; i < 1000; i++) { resultado += i; }

// ✓ Bien: StringBuilder es mutable, no recrea el String en cada paso
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) { sb.append(i); }
String resultado2 = sb.toString();
```

Con 3-4 concatenaciones sueltas, `+` es perfectamente legible y el compilador lo optimiza. El problema real aparece dentro de bucles con muchas vueltas — ahí sí importa `StringBuilder`.

4. Operadores

Tipo	Operadores
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Relacionales	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>
Lógicos	<code>&&</code> <code> </code> <code>!</code> (cortocircuito: si <code>a</code> ya decide el resultado, <code>b</code> ni se evalúa)

5. Estructuras de control

```
if (num % 2 == 0) {
    System.out.println("Par");
} else {
    System.out.println("Impar");
}

String tipoDia = switch (dia) { // switch moderno: expresión, sin break
    case 1, 7 -> "Finde";
    default -> "Laborable";
};
```

6. Bucles

```
for (int i = 0; i < 5; i++) { System.out.println(i); }
for (String s : lista) { System.out.println(s); }

int contador = 5;
while (contador > 0) { contador--; }

do { opcion = leerOpcion(); } while (opcion != 0); // se ejecuta AL MENOS una vez
```

`break` sale del bucle entero; `continue` se salta solo la vuelta actual y sigue con la siguiente. Con bucles anidados, una **etiqueta** (`busqueda:`)

permite que un `break` `busqueda`; desde el bucle interior rompa también el exterior, en vez de solo el de dentro.

7. Arrays

```
int[] numeros = {1, 2, 3};
numeros[0]; numeros.length;
int[][] matriz = {{1,2}, {3,4}};
```

Tamaño fijo. Si necesitas que crezca o encoja, esa es una **colección** (sección 17).

7.1 La clase de utilidades `Arrays`

```
int[] numeros = {5, 3, 1};
Arrays.sort(numeros); // ordena EL PROPIO array, no devuelve uno nuevo
System.out.println(Arrays.toString(numeros)); // [1, 3, 5] – un array normal no imprime bien con println directo
boolean igual = Arrays.equals(numeros, new int[]{1,3,5}); // == compararia referencias, no contenido
int[] copia = Arrays.copyOf(numeros, 5); // [1, 3, 5, 0, 0] – copia y rellena con el valor por defecto
```

`System.out.println(numeros)` a secas imprime algo como `[I@1b6d3586` — la representación por defecto de `Object` (sección 12), porque un array no sobrescribe `toString()`. Usa siempre `Arrays.toString()` para verlo de verdad.

8. Métodos (funciones)

```
public static int sumar(int a, int b) { return a + b; }
public static int sumarVarios(int... numeros) { // varargs
    int total = 0;
    for (int n : numeros) total += n;
    return total;
}
```

Java pasa parámetros **por valor**: con un objeto se copia la referencia (puedes modificar su contenido, no reasignar la variable original).

8.1 Sobrecarga de métodos (*overloading*)

```
public static int sumar(int a, int b) { return a + b; }
public static double sumar(double a, double b) { return a + b; } // mismo nombre, distintos parámetros
public static int sumar(int a, int b, int c) { return a + b + c; }
```

Varios métodos con el **mismo nombre** pero distinta lista de parámetros (en cantidad o en tipo). Java elige cuál ejecutar mirando los argumentos con los que lo llamas, en tiempo de compilación — no confundir con el *overriding* de la sección 11.2 (redefinir un método heredado), que se decide en tiempo de ejecución según el tipo real del objeto.

Parte 2 · Programación Orientada a Objetos

9. Modificadores de acceso, `static` y `final`

	Misma clase	Mismo paquete	Subclase (otro paquete)	Otro paquete
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
(default)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

Regla de oro: el nivel más restrictivo posible. Atributos `private`, métodos `public` solo si hace falta.

```
public class Contador {
    private static int total = 0; // UNA copia, compartida por todos
    public Contador() { total++; }
    public static int getTotal() { return total; } // static: sin objeto
}
```

`final` : variable que no se reasigna, método que no se sobreescribe, clase que no se hereda (`String` es `final`).

10. Clases, objetos, getters y setters

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.setEdad(edad);
    }

    public String getNombre() { return nombre; }

    public void setEdad(int edad) {
        if (edad >= 0) { this.edad = edad; }
    }
}
```

Getters/setters: evitan estados inválidos, no rompen código externo si cambias la implementación, y los usan frameworks como Spring por reflexión.

11. Los 4 pilares de la POO

11.1 Encapsulación

```
public class CuentaBancaria {
    private double saldo;
    public void retirar(double cantidad) {
        if (cantidad > saldo) throw new IllegalStateException("Saldo insuficiente");
        saldo -= cantidad;
    }
}
```

11.2 Herencia

```
public class Animal {
    protected String nombre;
    public void hacerSonido() { System.out.println("..."); }
}
public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("Guau"); }
}
```

Herencia simple de clases. Para "heredar" varias capacidades a la vez, interfaces (sección 13).

11.3 Polimorfismo

```
List<Animal> animales = List.of(new Perro(), new Gato());
for (Animal a : animales) { a.hacerSonido(); } // cada uno ejecuta SU versión
```

11.3.1 Casting e instanceof con pattern matching

A veces necesitas saber el tipo **concreto** detrás de una referencia polimórfica, para hacer algo que solo tiene sentido para ese tipo:

```
for (Animal a : animales) {
    if (a instanceof Perro perro) { // pattern matching (Java 16+): comprueba Y castea a la vez
        perro.buscarPelota(); // método que solo tiene Perro, no Animal
    }
}
```

Antes de Java 16 hacía falta escribir `if (a instanceof Perro) { Perro perro = (Perro) a; ... }` — dos pasos separados con el mismo cast repetido. Es exactamente el mismo mecanismo que usamos en `equals()` en la sección 12.1 (`if (!(o instanceof Punto p)) return false;`). Aun así, necesitar `instanceof` a menudo es señal de que te falta un método en la superclase o interfaz — si puedes resolverlo con polimorfismo puro (cada clase implementa su versión), casi siempre queda más limpio.

11.4 Abstracción

```
public abstract class Figura {
    public abstract double area();
}
public class Circulo extends Figura {
    private double radio;
    @Override
    public double area() { return Math.PI * radio * radio; }
}
```

12. equals(), hashCode() y toString()

Toda clase hereda de `Object`. La versión por defecto de `toString()` es fea (`Persona@1b6d3586`), y la de `equals()` es lo mismo que `==` (compara identidad, no contenido).

```
public class Persona {
    private String nombre;
    private int edad;

    @Override
    public String toString() {
        return "Persona{nombre=" + nombre + ", edad=" + edad + "}";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Persona otra = (Persona) o;
        return edad == otra.edad && Objects.equals(nombre, otra.nombre);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nombre, edad); // MISMOS campos que equals()
    }
}
```

El contrato de `equals()`: reflexivo, simétrico, transitivo, consistente, y `x.equals(null)` siempre `false`.

Regla de oro: si sobrescribes `equals()`, sobrescribe **SIEMPRE** `hashCode()` también, usando los mismos campos. Si no, un `HashSet/HashMap` puede "no encontrar" un objeto que según `equals()` ya estaba ahí — compila perfectamente y falla en silencio, uno de los bugs más traicioneros de Java.

En la práctica: IntelliJ los genera bien con `Alt+Insert` → Generate, y un `record` (sección 15) te los da gratis y coherentes.

12.1 El bug del HashSet, en vivo

Para que se quede grabado de verdad, así es como se ve el bug de la advertencia de arriba, paso a paso:

```
public class Punto {
    private int x, y;
    // ... constructor ...

    @Override
    public boolean equals(Object o) { // sobrescrito ✓
        if (this == o) return true;
        if (!(o instanceof Punto p)) return false;
```

```

        return x == p.x && y == p.y;
    }
    // hashCode() NO sobrescrito – sigue usando el de Object (basado en la dirección de memoria)
}

Set<Punto> visitados = new HashSet<>();
visitados.add(new Punto(3, 4));

visitados.contains(new Punto(3, 4)); // ✗ false – ¡aunque equals() diría true!

```

¿Por qué pasa esto? `HashSet` no compara con `equals()` contra **todos** los elementos — primero calcula el `hashCode()` del objeto que buscas y solo mira dentro del "cajón" (*bucket*) que le corresponde a ese hash. Como el `hashCode()` heredado de `Object` depende de la dirección de memoria, los dos `Punto(3,4)` —aunque iguales según tu `equals()` — acaban en cajones distintos, y el segundo nunca llega a compararse con el primero. No hay ningún error ni excepción: simplemente, silenciosamente, no lo encuentra. Es exactamente el tipo de bug que te puede hacer perder una tarde entera si no sabes qué buscar.

12.2 Interfaces con métodos `default` : el puente hacia la Parte 4

Una interfaz puede tener métodos con cuerpo, marcados `default`. Sirven para añadir comportamiento nuevo a una interfaz sin romper a quien ya la implementaba antes:

```

public interface Notificable {
    void enviar(String mensaje); // abstracto – lo implementa cada clase

    default void enviarUrgente(String mensaje) { // default – todas las clases lo heredan gratis
        enviar("URGENTE: " + mensaje);
    }
}

```

Esto es justo lo que hace posible que `Predicate`, `Function`, `Comparator` ... tengan métodos como `and()`, `andThen()` o `thenComparing()`: son interfaces funcionales (un único método **abstracto**) que además traen varios métodos `default` contruidos encima de ese único método. Le dedicamos toda la Parte 4 a explotar esta idea a fondo.

13. Interfaces vs. clases abstractas

```

public interface Medible {
    double medir();
}

public class Parcela implements Medible { // puedes implementar VARIAS interfaces a la vez
    @Override public double medir() { return 100.0; }
}

```

	Clase abstracta	Interfaz
Herencia/implementación	Solo una (<code>extends</code>)	Varias (<code>implements</code>)
Atributos	Sí, con estado propio	Solo constantes
Cuándo usarla	Base común, código compartido ("es un")	Una capacidad ("puede hacer X")

✗ Mal: la clase concreta

```
ArrayList<String> lista = new ArrayList<>();
```

✓ Bien: la interfaz

```
List<String> lista = new ArrayList<>();
```

Hay un tipo especial de interfaz que merece su propia parte entera: la **interfaz funcional**, la que le puedes pasar una lambda directamente. Le dedicamos toda la Parte 4 (secciones 18 a 30) porque es, de largo, lo que más se usa en código Java moderno — aquí solo nos quedamos con la idea general de interfaz.

14. Enums (enumeraciones)

```

public enum Direccion { NORTE, SUR, ESTE, OESTE }

public enum Planeta {
    MERCURIO(0.39), TIERRA(1.0);
    private final double distanciaUA;
    Planeta(double distanciaUA) { this.distanciaUA = distanciaUA; }
    public double getDistanciaUA() { return distanciaUA; }
}

```

Ventajas: valores imposibles de escribir mal, código legible, y el compilador avisa si te falta un caso en un `switch` exhaustivo.

15. Records

```
public record Punto(int x, int y) {}

Punto p = new Punto(3, 4);
p.x(); // 3
p; // Punto[x=3, y=4] al imprimirlo
```

Genera solo, y coherentes entre sí: constructor, getters, `equals()`, `hashCode()` y `toString()`. Úsalo para datos inmutables; para estado mutable y lógica, clase normal.

15.1 Records con validación y con interfaces

El "constructor compacto" de un `record` te deja validar sin repetir los parámetros, y un record puede implementar interfaces igual que cualquier clase:

```
public record Rango(int minimo, int maximo) implements Comparable<Rango> {
    public Rango { // constructor compacto: sin paréntesis, sin repetir "this.x = x"
        if (minimo > maximo) {
            throw new IllegalArgumentException("El mínimo no puede superar al máximo");
        }
    }
    @Override
    public int compareTo(Rango otro) { return Integer.compare(this.minimo, otro.minimo); }
}
```

Sigue siendo inmutable (no hay setters), sigue teniendo `equals()` / `hashCode()` / `toString()` gratis, y además ahora se puede ordenar en una lista con `Collections.sort()` o `.sorted()` sin pasarle un `Comparator` aparte.

15.2 Ejercicios de repaso rápido (Parte 1 y 2)

1. ¿Qué imprime este código, y por qué? `int a = 5; int b = a; b = 10; System.out.println(a);`

Solución: imprime `5`. Los primitivos se copian por valor: `b` es una copia independiente, cambiarla no afecta a `a`.

2. Tienes una clase `Rectangulo` que hereda de `Figura` (sección 11.4). ¿Qué modificador de acceso usarías para un método que solo deben poder llamar las subclases de `Figura`, nunca código de fuera del paquete?

Solución: `protected`.

3. ¿Por qué un `record Punto(int x, int y)` no necesita que tú escribas `equals()` a mano, pero una clase normal con los mismos campos sí?

Solución: el compilador genera automáticamente `equals()` / `hashCode()` / `toString()` para los records a partir de sus componentes; en una clase normal, sin sobrescribirlos, heredas la versión de `Object` (identidad, sección 12), que casi nunca es lo que quieres.

Parte 3 · Genéricos y colecciones

16. Genéricos (Generics)

`List<String>`: ese `<String>` es un **parámetro de tipo**. Antes de Java 5, las colecciones guardaban `Object` a secas: mezclabas tipos sin querer y el error explotaba en **ejecución** (`ClassCastException`), no en compilación.

```
List<String> lista = new ArrayList<>();
lista.add("Ana");
lista.add(42); // ✖ ERROR DE COMPILACIÓN, ni se ejecuta
String nombre = lista.get(0); // sin cast
```

Puedes escribir tus propias clases y métodos genéricos:

```
public class Caja<T> {
    private T contenido;
    public void guardar(T contenido) { this.contenido = contenido; }
    public T sacar() { return contenido; }
}

public static <T extends Number> double sumarTodos(List<T> numeros) { // tipo acotado
    double total = 0;
    for (T n : numeros) total += n.doubleValue();
    return total;
}
```

? (comodín): `List<? extends Number>` acepta cualquier lista de números para **leer**; `List<? super Integer>` permite **escribir** enteros.

Type erasure: la info de los genéricos solo existe al compilar; en ejecución `List<String>` y `List<Integer>` son la misma clase `List`. Por eso no puedes hacer `new T()` ni `instanceof List<String>`.

16.1 PECS: cuándo usar `extends` y cuándo `super`

La regla mnemotécnica que usan hasta los libros en inglés: **P**roducer **E**xtends, **C**onsumer **S**uper. Si una lista solo te va a dar datos (la vas a leer, "produce" valores para ti), usa `? extends T`. Si una lista solo va a recibir datos tuyos (la vas a rellenar, "consume" lo que le das), usa `? super T`.

```
public static void copiar(List<? extends Number> origen, List<? super Number> destino) {
    for (Number n : origen) { // origen: solo LEO, así que "extends" – puedo leer como Number con seguridad
        destino.add(n);      // destino: solo ESCRIBO, así que "super" – puedo meter un Number con seguridad
    }
}

List<Integer> enteros = List.of(1, 2, 3);
List<Object> destino = new ArrayList<>();
copiar(enteros, destino); // funciona: List<Integer> encaja en "? extends Number", List<Object> en "? super Number"
```

Con `? extends Number` el compilador te **prohíbe escribir** en `origen` (salvo `null`): si pudieras meter cualquier cosa, podrías romper la lista de otra persona sin que ella lo espere. Con `? super Number` el compilador te **prohíbe leer** con un tipo concreto (solo te deja leer como `Object`): no sabe si por debajo hay una `List<Number>`, una `List<Object>` o una `List<Serializable>`. El compilador no es tonto — te está protegiendo de errores en tiempo de ejecución.

17. Colecciones (List, Set, Map)

Colección	Orden	Duplicados
<code>List</code>	De inserción	Permite
<code>Set</code>	No garantizado	No permite
<code>Map</code>	Clave→valor	Claves únicas

```
List<String> nombres = new ArrayList<>();
nombres.add("Ana");

Set<Integer> unicos = new HashSet<>(List.of(1, 2, 2)); // {1, 2} – usa equals()/hashCode() (sección 12)

Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 20);
edades.getOrDefault("Pepe", 0);
```

```
for (Map.Entry<String, Integer> e : edades.entrySet()) {
    System.out.println(e.getKey() + " → " + e.getValue());
}
```

Declara siempre con el tipo interfaz (`List`, no `ArrayList`) — cambias de implementación sin tocar el resto del código.

17.1 `ArrayList` vs `LinkedList`, `HashMap` vs `TreeMap`

Implementación	Cuándo elegirla
<code>ArrayList</code>	Por defecto. Acceso por índice rápido; añadir al final, rápido.
<code>LinkedList</code>	Muchas inserciones/borrados en medio de la lista. Raro necesitarla en la práctica.
<code>HashMap</code> / <code>HashSet</code>	Por defecto. Rapidísimos, sin orden garantizado.
<code>LinkedHashMap</code> / <code>LinkedHashSet</code>	Como el Hash-, pero recuerdan el orden de inserción.
<code>TreeMap</code> / <code>TreeSet</code>	Necesitas las claves siempre ordenadas (usan <code>Comparable</code> o un <code>Comparator</code>).

Ejercicio: tienes `List<Integer> numeros = List.of(5, 3, 3, 1, 5)`. Sin bucles manuales, obtén cuántos valores **distintos** hay. Pista: mete la lista en un `Set` y mira su tamaño — recuerda que `Set` usa `equals()` / `hashCode()` (sección 12) para decidir qué es "distinto".

17.2 Recorrer y modificar a la vez: `Iterator`

Borrar elementos de una lista mientras la recorres con un `for-each` es un error clásico:

```
// ✗ ConcurrentModificationException en tiempo de ejecución
for (Alumno a : alumnos) {
    if (!a.aprobado()) { alumnos.remove(a); } // modificas la lista MIENTRAS la recorres
}

// ✓ Bien: Iterator explícito, con su propio remove() seguro
Iterator<Alumno> it = alumnos.iterator();
while (it.hasNext()) {
    Alumno a = it.next();
    if (!a.aprobado()) { it.remove(); } // seguro: el propio Iterator sabe que acaba de quitar uno
}

// ✓ Alternativa moderna, más corta:
alumnos.removeIf(a -> !a.aprobado()); // removeIf() recibe un Predicate<T> (sección 21)
```

El `for-each` (`for (Alumno a : alumnos)`) es, por dentro, exactamente un `Iterator` — solo que oculto. Por eso ambos fallan igual si intentas borrar sin pasar por el `Iterator` mismo.

17.3 `Queue` y `Deque` : colas para procesar en orden

```
Deque<String> pila = new ArrayDeque<>();
pila.push("a"); pila.push("b");
pila.pop(); // "b" – última en entrar, primera en salir (LIFO)

Queue<String> cola = new ArrayDeque<>();
cola.offer("a"); cola.offer("b");
cola.poll(); // "a" – primera en entrar, primera en salir (FIFO)
```

`ArrayDeque` sirve para ambos casos (pila y cola) y hoy se prefiere sobre la vieja clase `Stack`, que es más lenta por motivos históricos (sincronización que casi nunca hace falta).

17.4 Convertir entre `Array` y colecciones, e inmutabilidad

```
List<String> desdeArray = Arrays.asList("a", "b", "c"); // vista sobre el array: tamaño FIJO, no admite add()/remove()

List<String> immutable1 = List.of("a", "b"); // totalmente inmutable: ni add(), ni set(), nada
List<String> immutable2 = Collections.unmodifiableList(listaNormal); // "envoltorio" de solo lectura sobre otra lista

List<String> mutable = new ArrayList<>(immutable1); // copia mutable a partir de una inmutable
```

Con `Collections.unmodifiableList()`, si alguien conserva una referencia a la `listaNormal` original y la modifica, ese cambio **sí se refleja** en tu vista "inmutable" — solo bloquea la modificación a través de esa vista, no protege el contenido de raíz. `List.of()` no tiene ese problema porque copia los datos.

Parte 4 · Interfaces funcionales y programación funcional, a fondo

Esta es la parte gorda de estos apuntes. Las interfaces funcionales son, sin exagerar, lo que más vas a usar en Java moderno — cualquier stream, cualquier `forEach`, cualquier ordenación con `Comparator` pasa por aquí. Vamos despacio y con muchos ejemplos, porque merece la pena entenderlo bien una vez y no tener que volver a esto cada dos por tres.

18. Qué es una interfaz funcional (y por qué te vas a pasar la vida usándolas)

Una **interfaz funcional** es una interfaz normal, con un truco: tiene **un único método abstracto** (se llama SAM, de *Single Abstract Method*). Puede tener más métodos, pero con cuerpo (`default` o `static`); el que hay que implementar sí o sí, solo hay uno.

¿Por qué te importa? Porque esa regla de "solo un método" es justo lo que permite pasarle una **lambda** directamente, sin escribir una clase entera. Java sabe que si el tipo esperado es una interfaz funcional, tu lambda es literalmente el cuerpo de ese método.

```
@FunctionalInterface
interface Saludo {
    String saluda(String nombre);
}

Saludo s = nombre -> "Ey, " + nombre + "!";
System.out.println(s.saluda("Jorge")); // Ey, Jorge!
```

`@FunctionalInterface` no hace nada en ejecución, pero le dice al compilador "vigila que esta interfaz se quede con un solo método abstracto". Si un día añades por error un segundo método sin cuerpo, te lo marca en rojo antes de que sea un problema. Es opcional, pero úsala siempre en tus propias interfaces funcionales — es gratis.

Truco para reconocerlas: si en la documentación ves un parámetro de tipo `Runnable`, `Comparator<T>`, `Function<T,R>`... la respuesta casi siempre es "para pasarle una lambda".

18.1 Lo mismo, sin lambda: para que veas qué te estás ahorrando

Antes de Java 8, para pasar "un trozo de comportamiento" como argumento, la única opción era una **clase anónima**. Compara:

```
// ✗ Antes: clase anónima — mucho ruido para una línea de lógica real
Saludo s1 = new Saludo() {
    @Override
    public String saluda(String nombre) {
        return "Ey, " + nombre + "!";
    }
};

// ✓ Ahora: lambda — la misma idea, sin el envoltorio
Saludo s2 = nombre -> "Ey, " + nombre + "!";
```

Ambas hacen exactamente lo mismo en tiempo de ejecución. La diferencia es sintáctica y de legibilidad — pero cuando tienes streams con cinco o seis pasos encadenados, esa diferencia es la que hace que el código se pueda leer de un vistazo en vez de en diez pantallas.

18.2 Vista previa: las 4 interfaces que vas a usar el 90% del tiempo

Interfaz	Pregunta que responde	Sección
<code>Function<T,R></code>	"Dame esto, te devuelvo otra cosa"	20
<code>Predicate<T></code>	"Esto, ¿cumple la condición?"	21
<code>Consumer<T></code>	"Haz algo con esto, no me devuelvas nada"	22
<code>Supplier<T></code>	"Dame algo, no te doy nada a cambio"	22

Todo lo demás en esta parte (`BiFunction`, `UnaryOperator`, las especializaciones de `int` / `long` / `double` ...) son variaciones sobre estas cuatro ideas. Si en algún momento te pierdes, vuelve a esta tabla.

19. Lambdas a fondo: sintaxis, captura de variables y "this"

```
Runnable r = () -> System.out.println("Hola"); // sin parámetros

Consumer<String> c = s -> System.out.println(s); // 1 parámetro, paréntesis opcionales

Comparator<Integer> cmp = (a, b) -> a - b; // varios: paréntesis obligatorios

Function<Integer, Integer> doble = n -> n * 2; // expresión: sin llaves, sin return
```

```
Function<Integer, Integer> dobleConLog = n -> { // bloque: llaves + return explícito
    System.out.println("Calculando el doble de " + n);
    return n * 2;
};
```

19.1 Captura de variables: "effectively final"

Una lambda puede usar variables de fuera de su cuerpo (un *closure*). Regla estricta: esa variable debe ser **effectively final** — aunque no lleve `final` escrito, no la puedes reasignar después.

```
int contador = 0;
Runnable tarea = () -> System.out.println("Contador vale " + contador); // OK

contador++; // si esto va DESPUÉS, ni compila: "should be final or effectively final"
```

¿Por qué tan estricto? La lambda puede ejecutarse en otro momento, incluso en otro hilo. Si la variable pudiera cambiar mientras tanto, no habría forma de saber con qué valor se ejecutaría. Congelando el valor al crear la lambda, Java se ahorra ese lío.

19.2 "this" dentro de una lambda

Dentro de una lambda, `this` es el de la clase que la rodea, **no** el de la lambda (las lambdas no tienen su propio `this`). En una clase anónima, en cambio, `this` sí es el de la propia clase anónima — es la diferencia que más pilla por sorpresa a quien viene de clases anónimas.

No hace falta memorizar todos los detalles: para el día a día basta con saber que una lambda "vive" dentro de la clase que la rodea, no es su propio mundo aparte.

19.3 Tabla resumen de sintaxis

Parámetros	Sintaxis	Ejemplo
Cero	<code>() -> expr</code>	<code>() -> "hola"</code>
Uno (tipo inferido)	<code>p -> expr</code>	<code>n -> n * 2</code>
Uno (tipo explícito)	<code>(Tipo p) -> expr</code>	<code>(Integer n) -> n * 2</code>
Varios	<code>(p1, p2) -> expr</code>	<code>(a, b) -> a + b</code>
Cuerpo largo	<code>(p) -> { ...; return ...; }</code>	ver <code>dobleConLog</code> más arriba

Ejercicio: ¿por qué NO compila esto?

```
int base = 10;
List.of(1, 2, 3).forEach(n -> {
    base = base + n; // ...
});
```

Porque `base` se reasigna dentro de la lambda: deja de ser *effectively final*. Soluciones reales: un array de una celda (`int[] base = {10};`), o mejor todavía, no acumular a mano y usar `.reduce()` (sección 24).

20. `Function<T, R>` a fondo

Recibe un `T`, devuelve un `R`. Lo interesante es **encadenarlas**:

```
Function<Integer, Integer> sumarUno = n -> n + 1;
Function<Integer, Integer> multiplicarPorDos = n -> n * 2;

// andThen: ejecuta la primera, y con SU resultado la segunda
sumarUno.andThen(multiplicarPorDos).apply(3); // (3+1)*2 = 8

// compose: al revés, ejecuta la del paréntesis PRIMERO
sumarUno.compose(multiplicarPorDos).apply(3); // (3*2)+1 = 7
```

Truco para no confundirte: en `a.andThen(b)`, la **a va primero** (se lee en orden). En `a.compose(b)`, la **b va primero** (como en matemáticas, $f(g(x))$: g se ejecuta antes).

```
Function<String, String> limpiar = String::trim.andThen(String::toUpperCase);
limpiar.apply(" hola jorge "); // "HOLA JORGE"
```

20.1 Un caso real: pipeline de validación con `Function`

Un uso muy típico en apps reales: encadenar varias transformaciones de texto antes de guardarlo, sin escribir un método gigante con quince `if`.

```
Function<String, String> quitarEspacios = String::trim;
Function<String, String> aMinúsculas = String::toLowerCase;
Function<String, String> quitarTildesBasico = s -> s.replace("á","a").replace("é","e");
```

```
Function<String, String> normalizar = quitarEspacios
    .andThen(aMinusculas)
    .andThen(quitarTildesBasico);

String emailLimpio = normalizar.apply("  Jorge.Gonzalez@Correo.com  ");
```

Cada pieza es diminuta y se puede testear sola (sección 36). Si un día cambia una regla, tocas una línea, no el método entero.

Ejercicio: escribe un `Function<Integer, Integer>` llamado `cuadrado` y combínalo con `sumarUno` de arriba usando `andThen` para que, dado 3, calcule primero el cuadrado y luego sume uno. Resultado esperado: 10. (*sumarUno.compose(cuadrado)* también da el mismo resultado aquí — piensa por qué.)

21. `Predicate<T>` a fondo

Con `and()`, `or()` y `negate()` combinas condiciones pequeñas y con nombre en vez de una lambda gigante e ilegible.

```
Predicate<Integer> esPar = n -> n % 2 == 0;
Predicate<Integer> esPositivo = n -> n > 0;

Predicate<Integer> esParYPositivo = esPar.and(esPositivo);
esParYPositivo.test(4); // true
esParYPositivo.test(-4); // false
```

```
Predicate<Alumno> aprobado = Alumno::aprobado;
Predicate<Alumno> notaAlta = a -> a.nota() >= 8;

List<Alumno> matriculaDeHonor = alumnos.stream()
    .filter(aprobado.and(notaAlta))
    .toList();
```

También existe `Predicate.isEqual(objeto)`, un predicado que compara con `equals()` (sección 12) contra un valor fijo.

21.1 Filtrado real con varios predicados combinados

```
Predicate<Alumno> tieneNombre = a -> a.nombre() != null && !a.nombre().isBlank();
Predicate<Alumno> suspenso = notaAlta.negate(); // negate(): lo contrario del predicado

List<Alumno> recuperacion = alumnos.stream()
    .filter(tieneNombre.and(aprobado.negate()).or(suspenso))
    .toList();
```

Es fácil liarse con `negate()`: `a.negate()` no modifica `a` (los predicados son inmutables), **devuelve uno nuevo**. Si esperabas que cambiara el original y no reasignas la variable, sigues usando el predicado de siempre sin darte cuenta.

Ejercicio: con los `Predicate` de esta sección, escribe uno llamado `esAdmisible` que sea verdadero cuando el alumno está aprobado o tiene una nota mayor que 4 y algún tipo de necesidad especial marcada (asume un método `a.tieneAdaptacion()`). Pista: vas a necesitar `or()` combinando dos condiciones hechas con `and()`.

22. `Consumer<T>` y `Supplier<T>` a fondo

22.1 `Consumer<T>`: encadenar acciones con `andThen()`

```
Consumer<String> imprimir = s -> System.out.println("Valor: " + s);
Consumer<String> log = s -> System.out.println("[LOG] " + s);

Consumer<String> ambos = imprimir.andThen(log); // haz esto, Y ADEMÁS esto otro, con el mismo dato
ambos.accept("Jorge");
```

22.2 `Supplier<T>`: pereza a propósito (lazy evaluation)

Lo importante no es solo que "genera un valor", sino que **retrasa** el trabajo hasta que hace falta de verdad. Es la base de `Optional.orElseGet()`:

```
// MAL: crearPorDefecto() se ejecuta SIEMPRE, exista o no el alumno,
// porque el argumento se evalúa antes de llamar al método
Alumno r1 = alumno.orElse(crearPorDefecto());

// BIEN: con Supplier, SOLO se ejecuta si hace falta
Alumno r2 = alumno.orElseGet(() -> crearPorDefecto());
```

Si `crearPorDefecto()` consulta una base de datos o llama a una API, la diferencia es real, no solo estilo.

22.4 `Optional<T>`: evitar el `null` a propósito

`Optional<T>` es una "caja" que puede contener un valor o estar vacía — una forma explícita de decir "esto puede no existir", en vez de

devolver `null` y confiar en que quien lo recibe se acuerde de comprobarlo.

```
Optional<Alumno> buscar(String nombre) {
    return alumnos.stream()
        .filter(a -> a.nombre().equals(nombre))
        .findFirst(); // findFirst() ya devuelve Optional<Alumno>
}

// Formas de consumir un Optional, de más a menos recomendadas:
buscar("Ana").ifPresent(a -> System.out.println(a.nombre())); // Consumer: solo actúa si hay valor

Alumno a1 = buscar("Ana").orElse(Alumno.VACIO); // valor por defecto ya calculado
Alumno a2 = buscar("Ana").orElseGet(() -> crear("Ana")); // Supplier: perezoso (sección 22.2)
Alumno a3 = buscar("Ana").orElseThrow(() -> new AlumnoNoEncontradoException("Ana"));

if (buscar("Ana").isPresent()) { // ✗ evita esto: vuelve a caer en el estilo "if + null"
    Alumno a = buscar("Ana").get(); // y además busca DOS veces
}
```

Llamar a `.get()` sobre un `Optional` vacío lanza `NoSuchElementException` — es tan peligroso como no comprobar un `null`. La idea de `Optional` es precisamente **no** usar `.get()` a pelo; usa `.map()`, `.ifPresent()`, `.orElse()` / `orElseGet()` / `orElseThrow()` en su lugar.

`Optional` también tiene su propio `map()`, que es un `Function` aplicado solo si hay valor: `buscar("Ana").map(Alumno::nota).orElse(0.0)` — si no hay alumno, ni siquiera se intenta leer `.nota()`, evitando el `NullPointerException` de raíz.

22.3 Un `Supplier` con caché (memoización sencilla)

```
public class Cache<T> {
    private T valor;
    private boolean calculado = false;
    private final Supplier<T> generador;

    public Cache(Supplier<T> generador) { this.generador = generador; }

    public T get() {
        if (!calculado) {
            valor = generador.get(); // solo se ejecuta la PRIMERA vez
            calculado = true;
        }
        return valor;
    }
}

Cache<List<Alumno>> alumnosCache = new Cache<>(() -> cargarDesdeBaseDeDatos());
```

Aquí `Supplier<T>` es literalmente "una receta para conseguir un `T` cuando yo diga", desacoplada de cuándo o cuántas veces se ejecuta esa receta. Es el mismo principio que `orElseGet()`, llevado a una clase propia.

Ejercicio: ¿qué pasaría si en vez de un `Supplier<T> generador` el constructor recibiera directamente un `T valor` ya calculado? Piensa en `cargarDesdeBaseDeDatos()` como una consulta lenta — ¿en qué momento se pagaría ese coste en cada versión?

23. Versiones de dos argumentos: `BiFunction`, `BiPredicate`, `BiConsumer`

Interfaz	Método
<code>BiFunction<T, U, R></code>	<code>R apply(T, U)</code>
<code>BiPredicate<T, U></code>	<code>boolean test(T, U)</code>
<code>BiConsumer<T, U></code>	<code>void accept(T, U)</code>

```
Map<String, Integer> edades = Map.of("Ana", 20, "Luis", 17);

BiConsumer<String, Integer> imprimir = (nombre, edad) ->
    System.out.println(nombre + " tiene " + edad + " años");

edades.forEach(imprimir); // Map.forEach() pide justo un BiConsumer<K, V>
```

No existe un `BiSupplier`: un `Supplier` no recibe nada, así que no hay "dos entradas" que darle.

23.1 `BiFunction` con `Map.computeIfAbsent` y `merge`

```
Map<String, List<Alumno>> porClase = new HashMap<>();

for (Alumno a : alumnos) {
    porClase.computeIfAbsent(a.clase(), clave -> new ArrayList<>())
        .add(a);
}
```

```
// merge() es un BiFunction<V,V,V> puro: combina el valor viejo y el nuevo
Map<String, Integer> contador = new HashMap<>();
for (Alumno a : alumnos) {
    contador.merge(a.clase(), 1, Integer::sum); // (viejo, nuevo) -> viejo + nuevo
}
```

`computeIfAbsent` recibe un `Function<K,V>` (solo necesita la clave para crear el valor si falta); `merge` recibe un `BiFunction<V,V,V>` (necesita el valor viejo Y el nuevo para decidir cómo combinarlos). Cada método del `Map` pide justo la forma que necesita, ni más ni menos — por eso hay tantas interfaces distintas en vez de una sola genérica.

Ejercicio: reescribe el bucle de `contador` usando `Collectors.groupingBy(Alumno::clase, Collectors.counting())` sobre el stream de `alumnos`, sin bucle `for` manual.

24. `UnaryOperator<T>` y `BinaryOperator<T>`

Casos particulares: `UnaryOperator<T>` es un `Function<T, T>` con entrada y salida del mismo tipo; `BinaryOperator<T>` es un `BiFunction<T, T, T>` igual de "cerrado".

```
BinaryOperator<Integer> max = (a, b) -> a > b ? a : b;

int suma = List.of(1,2,3,4).stream()
    .reduce(0, (acumulado, actual) -> acumulado + actual); // esto es un BinaryOperator<Integer>
```

¿Por qué no usar `Function<T,T>` directamente? Podrías, pero el nombre específico es más legible — y es literalmente lo que pide `.reduce()`.

24.1 Más ejemplos de `reduce()`

```
Optional<Integer> maximo = List.of(4,9,2).stream()
    .reduce(Integer::max); // sin valor inicial: devuelve Optional<T>, por si la lista está vacía

String frase = List.of("Hola", "que", "tal").stream()
    .reduce("", (acumulado, palabra) -> acumulado.isEmpty() ? palabra : acumulado + " " + palabra);
```

`.reduce(identidad, operador)` siempre devuelve el tipo directamente porque hay un valor de partida garantizado; `.reduce(operador)` sin identidad devuelve `Optional<T>` porque, si la lista está vacía, sencillamente no hay nada que devolver.

Ejercicio: usando un `BinaryOperator<Integer>`, escribe a mano el equivalente a `Integer::min` con una lambda `(a, b) -> ...`, y compruébalo con `reduce()` sobre `List.of(7, 2, 9, 1)`. Resultado esperado: 1.

25. Especializaciones primitivas: evitar el autoboxing

Cada `int` metido donde se espera `Integer` crea un objeto por debajo (sección 2). En un stream de millones de elementos, eso pesa. Por eso existen versiones especializadas para `int`, `long`, `double`.

Genérica (con boxing)	Especializada	Qué cambia
<code>Function<Integer,Integer></code>	<code>IntUnaryOperator</code>	<code>applyAsInt(int)</code> , sin cajas
<code>Function<Integer,R></code>	<code>IntFunction<R></code>	Entra <code>int</code> , sale objeto
<code>Function<T,Integer></code>	<code>ToIntFunction<T></code>	Entra objeto, sale <code>int</code>
<code>Predicate<Integer></code>	<code>IntPredicate</code>	<code>test(int)</code>
<code>Supplier<Integer></code>	<code>IntSupplier</code>	<code>getAsInt()</code>

```
IntStream.rangeClosed(1, 5)
    .map(n -> n * n) // aquí "map" espera un IntUnaryOperator, no Function<Integer,Integer>
    .forEach(System.out::println);
```

No hace falta memorizarlas todas: el patrón es `To<Tipo>` cuando el primitivo es la salida, `<Tipo>` al principio cuando es la entrada. Con eso deduces el resto.

25.1 Tabla completa con las variantes Long y Double

Familia	int	long	double
Entrada primitiva	<code>IntFunction<R></code>	<code>LongFunction<R></code>	<code>DoubleFunction<R></code>
Salida primitiva	<code>ToIntFunction<T></code>	<code>ToLongFunction<T></code>	<code>ToDoubleFunction<T></code>
Predicate	<code>IntPredicate</code>	<code>LongPredicate</code>	<code>DoublePredicate</code>
Consumer	<code>IntConsumer</code>	<code>LongConsumer</code>	<code>DoubleConsumer</code>
Supplier	<code>IntSupplier</code>	<code>LongSupplier</code>	<code>DoubleSupplier</code>

UnaryOperator	IntUnaryOperator	LongUnaryOperator	DoubleUnaryOperator
BinaryOperator	IntBinaryOperator	LongBinaryOperator	DoubleBinaryOperator

```
double media = alumnos.stream()
    .mapToDouble(Alumno::nota) // mapToDouble espera un ToDoubleFunction<Alumno>
    .average()
    .orElse(0.0);
```

Regla práctica: si usas `.mapToInt()`, `.mapToLong()` o `.mapToDouble()` en un stream, por dentro casi siempre te está pidiendo alguna de estas especializaciones — no hace falta buscarla a propósito, aparece sola al escribir el código.

26. Otras interfaces funcionales de toda la vida: `Runnable`, `Callable`, `Comparator`

Existen desde antes de las lambdas, pero cumplen el mismo SAM, así que hoy se usan igual.

```
Runnable tarea = () -> System.out.println("Ejecutando...");
new Thread(tarea).start();

Callable<Integer> conResultado = () -> { // devuelve algo Y puede lanzar checked
    Thread.sleep(100);
    return 42;
};
```

`Comparator<T>` es la más usada de las tres, sobre todo para ordenar:

```
alumnos.sort(Comparator.comparingDouble(Alumno::nota)
    .thenComparing(Alumno::nombre) // por nota, y si empatan, por nombre
    .reversed());                // de mayor a menor
```

Mismo espíritu que `Function`/`Predicate`: métodos default que combinan piezas pequeñas en algo más complejo, en vez de un método gigante lleno de `if`.

26.1 `Comparator` con varios criterios encadenados (ejemplo completo)

```
record Producto(String nombre, String categoria, double precio) {}

List<Producto> ordenados = productos.stream()
    .sorted(Comparator.comparing(Producto::categoria) // 1º por categoría
        .thenComparing(Comparator.comparingDouble(Producto::precio).reversed())) // 2º por precio desc
    .toList();
```

`Callable<V>` se usa sobre todo con hilos que devuelven resultado (`ExecutorService.submit(callable)` devuelve un `Future<V>`); si no necesitas ni resultado ni excepciones checked, `Runnable` basta y es más simple.

```
ExecutorService pool = Executors.newFixedThreadPool(2);

Callable<Integer> tarea = () -> {
    Thread.sleep(500);
    return 42;
};

Future<Integer> futuro = pool.submit(tarea); // no bloquea: la tarea corre en otro hilo
System.out.println("Haciendo otras cosas mientras...");
int resultado = futuro.get(); // AQUÍ sí espera, hasta que el Callable termine

pool.shutdown();
```

La diferencia clave frente a `Thread` + `Runnable` a pelo: un `ExecutorService` gestiona un grupo de hilos reutilizables por ti (no creas un `Thread` nuevo cada vez, que es costoso), y `Future<V>` te da un sitio donde recoger el resultado cuando esté listo, sin tener que organizar tú mismo la comunicación entre hilos.

Ejercicio: ordena una lista de `Alumno` primero por si están aprobados (los aprobados antes) y, dentro de cada grupo, por nombre alfabético. Pista: `Comparator.comparing(Alumno::aprobado).reversed().thenComparing(Alumno::nombre)` — piensa por qué hace falta invertir el primer comparador si `boolean` ordena `false` antes que `true`.

27. Referencias a métodos a fondo: los 4 tipos

Tipo	Sintaxis	Ejemplo	Lambda equivalente
1. Método estático	<code>Clase::metodo</code>	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
2. Instancia de objeto CONCRETO	<code>obj::metodo</code>	<code>System.out::println</code>	<code>x -> System.out.println(x)</code>
3. Instancia de tipo ARBITRARIO	<code>Clase::metodo</code>	<code>String::length</code>	<code>s -> s.length()</code>

4. Constructor	Clase::new	Alumno::new	(n, x) -> new Alumno(n, x)
----------------	------------	-------------	----------------------------

El tipo 3 es el que más líia: `String::length` funciona como `Function<String, Integer>` porque Java usa el **primer parámetro** como si fuera el objeto sobre el que se llama (`s.length()`).

```
BiFunction<String, Double, Alumno> crear = Alumno::new; // tipo 4, constructor de 2 parámetros
crear.apply("Ana", 9.0);
```

Si la lambda solo llama a un método que ya existe, usa la referencia (más corta y clara). Si hace algo más, lambda normal.

27.1 El caso trampa: métodos sobrecargados

Si una clase tiene varios métodos con el mismo nombre (sobrecarga), `Clase::metodo` por sí sola no basta para que Java sepa cuál usar — necesita el **contexto del tipo funcional esperado** para elegir la sobrecarga correcta.

```
class Calculadora {
    static int sumar(int a, int b) { return a + b; }
    static double sumar(double a, double b) { return a + b; }
}

BinaryOperator<Integer> sumaEnteros = Calculadora::sumar; // elige la versión int por el tipo del BinaryOperator
```

Si el compilador no tiene suficiente contexto (por ejemplo, con `var` sin más), la referencia a método es ambigua y toca ser explícito con una lambda o indicar el tipo.

Ejercicio: de los 4 tipos de referencia a método, ¿cuál es `alumno::compareTo` si `alumno` es una variable ya creada de tipo `Alumno` ? (Pista: es el tipo 2 — instancia de objeto concreto — aunque a primera vista, por el parámetro que recibe `compareTo` , parezca de otro tipo.)

28. Streams de principio a fin: un pipeline grande con todo

Juntamos casi todo lo de esta parte en un ejemplo real, con empleados — es donde de verdad se ve por qué merece la pena todo este rollo.

```
record Empleado(String nombre, String departamento, double salario) {}

List<Empleado> empleados = List.of(
    new Empleado("Ana", "IT", 2200),
    new Empleado("Luis", "IT", 1900),
    new Empleado("Marta", "Ventas", 2100)
);

Predicate<Empleado> deIT = e -> e.departamento().equals("IT");
Predicate<Empleado> salarioAlto = e -> e.salario() > 2000;

List<String> nombres = empleados.stream()
    .filter(deIT.and(salarioAlto)) // Predicate combinado
    .sorted(Comparator.comparingDouble(Empleado::salario).reversed()) // Comparator
    .map(Empleado::nombre) // Function (method reference)
    .toList();
System.out.println(nombres); // [Ana]

double totalSalarios = empleados.stream()
    .map(Empleado::salario)
    .reduce(0.0, Double::sum); // Double::sum es un BinaryOperator<Double>

empleados.stream()
    .filter(salarioAlto)
    .forEach(e -> System.out.printf("%s: %.0f€\n", e.nombre(), e.salario())); // Consumer

Map<String, List<Empleado>> porDepartamento = empleados.stream()
    .collect(Collectors.groupingBy(Empleado::departamento));
```

Cada método del stream (`.filter()` , `.map()` , `.sorted()` , `.reduce()` , `.forEach()`) recibe, por dentro, una interfaz funcional como parámetro. No es "otro tema" separado de los streams: es el mecanismo que los hace funcionar.

28.2 Chuleta de Streams: métodos intermedios y terminales

Un stream se construye con cero o más métodos **intermedios** (devuelven otro stream, se pueden encadenar) y termina con exactamente un método **terminal** (devuelve un valor final y "consume" el stream — no se puede reutilizar después).

Método	Tipo	Qué hace
<code>filter(Predicate)</code>	Intermedio	Se queda solo con los elementos que cumplen la condición
<code>map(Function)</code>	Intermedio	Transforma cada elemento en otro
<code>flatMap(Function)</code>	Intermedio	Como <code>map</code> , pero aplana streams anidados en uno solo
<code>sorted()</code> / <code>sorted(Comparator)</code>	Intermedio	Ordena los elementos
<code>distinct()</code>	Intermedio	Quita duplicados (usa <code>equals()</code> , sección 12)

<code>limit(n) / skip(n)</code>	Intermedio	Se queda con los <code>n</code> primeros / se salta los <code>n</code> primeros
<code>peek(Consumer)</code>	Intermedio	"Espía" cada elemento sin transformarlo — solo para depurar
<code>forEach(Consumer)</code>	Terminal	Ejecuta una acción por cada elemento, sin devolver nada
<code>collect(Collector)</code>	Terminal	Reúne el resultado en una lista, mapa, string...
<code>reduce(...)</code>	Terminal	Combina todos los elementos en uno solo (sección 24)
<code>count()</code>	Terminal	Cuenta cuántos elementos hay
<code>anyMatch / allMatch / noneMatch(Predicate)</code>	Terminal	Comprueba si alguno / todos / ninguno cumple la condición
<code>findFirst() / findAny()</code>	Terminal	Devuelve un <code>Optional</code> con el primer elemento (o cualquiera)
<code>toList()</code>	Terminal	Atajo moderno equivalente a <code>.collect(Collectors.toList())</code>

Un stream solo se puede recorrer **una vez**. Si guardas un `Stream<T>` en una variable y llamas a dos métodos terminales sobre él, el segundo lanza `IllegalStateException`. Si necesitas recorrerlo dos veces, vuelve a crear el stream desde la colección original (`.stream()` otra vez).

28.1 Segundo pipeline: agrupar, contar y unir texto

```
Map<Boolean, List<Empleado>> divididos = empleados.stream()
    .collect(Collectors.partitioningBy(salarioAlto)); // true/false según el Predicate

Map<String, Long> empleadosPorDepartamento = empleados.stream()
    .collect(Collectors.groupingBy(Empleado::departamento, Collectors.counting()));

String listado = empleados.stream()
    .map(Empleado::nombre)
    .collect(Collectors.joining(", ", "Empleados: [", "]"));
// "Empleados: [Ana, Luis, Marta]"

Map<String, double[]> nominaPorNombre = empleados.stream()
    .collect(Collectors.toMap(Empleado::nombre, e -> new double[]{e.salario()}));
```

`partitioningBy` es un caso particular de agrupar: siempre en exactamente dos grupos (`true` / `false`), a partir de un `Predicate`. `groupingBy` con un segundo argumento (`Collectors.counting()`, `Collectors.averagingDouble()` ...) no solo agrupa, sino que además **reduce** cada grupo a un solo valor.

28.3 Chuleta de `Collectors` más usados

Collector	Devuelve	Para qué
<code>Collectors.toList() / .toSet()</code>	<code>List<T></code> / <code>Set<T></code>	Reunir en una colección (hoy, mejor <code>.toList()</code> directo del stream)
<code>Collectors.joining(sep)</code>	<code>String</code>	Unir strings con un separador (y prefijo/sufijo opcionales)
<code>Collectors.counting()</code>	<code>Long</code>	Contar elementos (normalmente dentro de <code>groupingBy</code>)
<code>Collectors.summingInt/Double(Function)</code>	<code>Integer</code> / <code>Double</code>	Sumar un campo numérico
<code>Collectors.averagingDouble(Function)</code>	<code>Double</code>	Media de un campo numérico
<code>Collectors.minBy/maxBy(Comparator)</code>	<code>Optional<T></code>	El elemento mínimo/máximo según el comparador
<code>Collectors.groupingBy(Function)</code>	<code>Map<K, List<T>></code>	Agrupar por una clave
<code>Collectors.groupingBy(Function, Collector)</code>	<code>Map<K, R></code>	Agrupar Y reducir cada grupo (contar, sumar, etc.)
<code>Collectors.partitioningBy(Predicate)</code>	<code>Map<Boolean, List<T>></code>	Dividir en dos grupos según una condición
<code>Collectors.toMap(keyFn, valueFn)</code>	<code>Map<K,V></code>	Construir un mapa a partir del stream

29. Errores comunes y buenas prácticas con lambdas

29.1 Excepciones checked dentro de una lambda

Las interfaces de `java.util.function` no declaran `throws`. Si dentro de una lambda llamas a algo que lanza una excepción *checked* (sección 31), no compila.

```
// ✓ Solución típica: capturar dentro y relanzar como unchecked
Function<String, Date> parsear = texto -> {
    try {
        return formato.parse(texto);
    } catch (ParseException e) {
        throw new RuntimeException("Fecha con formato incorrecto: " + texto, e);
    }
};
```

29.2 No abuses: cuándo NO usar una lambda

- Más de 4-5 líneas: mejor un método con nombre y llamarlo con `Clase::metodo` .
- Si vas a poner breakpoints (sección 37), un método con nombre se depura mejor que una lambda anónima metida en medio de un stream.
- Evita efectos secundarios raros dentro de una lambda (modificar variables externas): rompe la idea de "función pura" y hace el código difícil de razonar.

Una lambda no es "más profesional" por sí misma que un `for` de toda la vida. Cuando el código quede más claro con streams, adelante; cuando quede más claro con un bucle normal, usa el bucle. Legibilidad por encima de postureo.

29.3 Más errores típicos

- **Capturar variables mutables por accidente:** si dentro de un bucle creas una lambda que usa la variable del bucle, en Java moderno cada iteración de un `for` con variable propia tiene su copia "effectively final" — conviene igualmente revisarlo si algo no se comporta como esperas.
- **Lambdas que devuelven `null` donde se espera un `Optional`** : si tu `Function<T,R>` puede no tener resultado, mejor que devuelva `Optional<R>` explícitamente que un `null` silencioso que explota tres llamadas después.
- **Sobre-anidar streams:** un `.map()` que dentro llama a otro stream entero suele poderse aplanar con `.flatMap()` . Si tu pipeline necesita más de 4-5 líneas de indentación, es señal de extraer un método con nombre.

Ejercicio: encuentra el bug — este código no compila. ¿Por qué?

```
Function<Integer, Integer> incrementar;
for (int i = 0; i < 3; i++) {
    incrementar = n -> n + i; // ...
}
```

Porque `incrementar` se reasigna en cada vuelta del bucle (no es *effectively final*) y además usa `i` , que también cambia. Arréglalo declarando la lambda dentro del bucle con una variable nueva en cada vuelta, o guardando cada una en una lista.

30. Chuleta final: todas las interfaces de `java.util.function`

Interfaz	Método	Para qué
<code>Function<T,R></code>	<code>R apply(T)</code>	Transformar
<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>	Transformar, dos entradas
<code>UnaryOperator<T></code>	<code>T apply(T)</code>	Transformar, mismo tipo
<code>BinaryOperator<T></code>	<code>T apply(T,T)</code>	Combinar (reduce)
<code>Predicate<T></code>	<code>boolean test(T)</code>	Preguntar sí/no
<code>BiPredicate<T,U></code>	<code>boolean test(T,U)</code>	Preguntar sí/no, dos entradas
<code>Consumer<T></code>	<code>void accept(T)</code>	Hacer algo, sin devolver
<code>BiConsumer<T,U></code>	<code>void accept(T,U)</code>	Hacer algo, dos entradas
<code>Supplier<T></code>	<code>T get()</code>	Generar bajo demanda
<code>Runnable</code>	<code>void run()</code>	Ejecutar, sin entrada ni salida
<code>Callable<V></code>	<code>V call()</code>	Como Runnable, devuelve y puede lanzar checked
<code>Comparator<T></code>	<code>int compare(T,T)</code>	Ordenar
<code>IntFunction / ToIntFunction / IntPredicate / IntConsumer / IntSupplier / IntUnaryOperator / IntBinaryOperator</code>	—	Como las de arriba sin autoboxing (y sus hermanas Long/Double)

Memoriza solo cuatro nombres (Function, Predicate, Consumer, Supplier) y el patrón "Bi-" (dos entradas) / "-Operator" (mismo tipo). El resto de la

tabla se deduce con lógica.

30.1 Ejercicios finales resueltos: combinando varias interfaces

Ejercicio A — validador de formulario. Dada una lista de `String` con nombres de usuario, quedarte solo con los válidos (no vacíos, sin espacios, longitud entre 3 y 15) y devolverlos en mayúsculas, ordenados alfabéticamente.

```
Predicate<String> noVacio = s -> s != null && !s.isBlank();
Predicate<String> sinEspacios = s -> !s.contains(" ");
Predicate<String> longitudValida = s -> s.length() >= 3 && s.length() <= 15;

List<String> usuariosValidos = usuarios.stream()
    .filter(noVacio.and(sinEspacios).and(longitudValida))
    .map(String::toUpperCase)
    .sorted()
    .toList();
```

Ejercicio B — resumen de pedidos. Con un `record Pedido(String cliente, double importe, boolean pagado)`, calcula el importe total cobrado y cuántos pedidos tiene cada cliente.

```
record Pedido(String cliente, double importe, boolean pagado) {}

Predicate<Pedido> pagado = Pedido::pagado;

double totalCobrado = pedidos.stream()
    .filter(pagado)
    .mapToDouble(Pedido::importe)
    .sum();

Map<String, Long> pedidosPorCliente = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::cliente, Collectors.counting()));
```

Fíjate en cuántas interfaces funcionales distintas hay escondidas en apenas seis líneas: un `Predicate<Pedido>` (referencia a método), un `ToDoubleFunction<Pedido>` dentro de `mapToDouble`, y un `Function<Pedido, String>` más un `Collector` dentro de `groupingBy`. Esto es, literalmente, cómo se ve el Java "de verdad" hoy en día — y a estas alturas ya lo entiendes entero, pieza por pieza.

30.2 Preguntas frecuentes sobre interfaces funcionales

¿Puedo guardar una lambda en una variable y reutilizarla varias veces?

Sí, sin problema — una vez asignada a una variable de un tipo funcional (`Function<T,R> f = ...`), es un objeto normal como cualquier otro. Puedes pasarla como argumento, guardarla en una lista, devolverla desde un método...

¿Una lambda puede lanzar excepciones?

Puede lanzar *unchecked* (`RuntimeException` y sus hijas) sin ningún problema. Las *checked* (sección 31) solo si la interfaz funcional que estás implementando las declara en su método (como `Callable<V>`, que declara `throws Exception`) — las interfaces normales de `java.util.function` no las declaran, por eso hay que envolverlas (sección 29.1).

¿Por qué a veces el compilador se queja de "target-type" o "no se puede inferir el tipo"?

Una lambda no tiene tipo por sí sola: su tipo lo decide el contexto (a qué variable o parámetro se está asignando). Si ese contexto es ambiguo — por ejemplo, un método sobrecargado que acepta tanto un `Runnable` como un `Callable<V>` — el compilador no sabe cuál de los dos quieres y te lo pide explícito.

¿Las interfaces funcionales solo sirven para streams?

No, para nada: `Comparator` existe desde Java 1.2, y librerías enteras (por ejemplo, callbacks de eventos en interfaces gráficas, o handlers en frameworks web) se apoyan en el mismo patrón SAM. Los streams son, simplemente, donde más se ven todas juntas a la vez.

¿Puedo crear mi propia interfaz funcional con genéricos?

Sí — de hecho combinar genéricos (Parte 3) con interfaces funcionales (Parte 4) es exactamente cómo están construidas `Function<T,R>` y compañía por dentro.

```
@FunctionalInterface
interface Transformador<E, S> {
    S transformar(E entrada);
}

Transformador<String, Integer> longitud = String::length;
```

En la práctica esto es literalmente reinventar `Function<T,R>`, así que en código real usarías la de `java.util.function` directamente — pero entender que **puedes** hacerlo te confirma que no hay magia, solo interfaces normales con una regla extra (una única abstracta).

¿Qué gano exactamente frente a escribir el `for` de siempre?

Nada "gratis" en rendimiento — de hecho un `for` simple suele ser algo más rápido que un stream equivalente para colecciones pequeñas. Lo que ganas es **composición**: piezas pequeñas, con nombre, que reutilizas y combinas (`and`, `andThen`, `negate` ...), en vez de un bloque de lógica monolítico que solo sirve para ese caso concreto.

Parte 5 · Excepciones, ficheros y diseño

31. Excepciones (manejo de errores)

```
try {
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: división por cero");
} finally {
    System.out.println("Esto se ejecuta SIEMPRE");
}
```

Excepción	Cuándo salta
<code>NullPointerException</code>	Usas un objeto <code>null</code>
<code>IOException</code>	Errores de archivos
<code>NumberFormatException</code>	<code>parseInt()</code> con texto no numérico

Checked (`Exception` : `IOException`) obligan a gestionar o declarar con `throws` . **Unchecked** (`RuntimeException`) no obligan a nada.

```
public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(String mensaje) { super(mensaje); }
}
```

31.1 Tu propia jerarquía de excepciones

```
public class AppException extends RuntimeException { // unchecked: base común de la app
    public AppException(String mensaje) { super(mensaje); }
}
public class AlumnoNoEncontradoException extends AppException {
    public AlumnoNoEncontradoException(int id) { super("No existe el alumno con id " + id); }
}
```

Tener una excepción base propia (`AppException`) te permite capturar "cualquier error de mi aplicación" en un solo `catch` en la capa más alta (el `Controller` , sección 34), mientras que en capas internas lanzas la excepción más específica posible.

try con múltiples catch: `catch (IOException | SQLException e)` captura ambas con el mismo bloque cuando el tratamiento es idéntico.

31.2 Excepciones con datos extra, no solo un mensaje

Una excepción es una clase como cualquier otra: puede tener sus propios campos, útiles para que quien la capture sepa exactamente qué pasó, no solo que "algo" pasó.

```
public class NotaInvalidaException extends Exception {
    private final double notaRecibida;

    public NotaInvalidaException(double notaRecibida) {
        super("Nota fuera de rango: " + notaRecibida);
        this.notaRecibida = notaRecibida;
    }
    public double getNotaRecibida() { return notaRecibida; } // quien capture puede consultarla
}

try {
    validarNota(15.0);
} catch (NotaInvalidaException e) {
    System.out.println("Nota rechazada: " + e.getNotaRecibida()); // no solo el mensaje de texto
}
```

Esto es justo lo que hace el `record Alumno` de tu proyecto de ejemplo "Colecciones-Streams-Excepciones-Ficheros": la validación en el constructor compacto (sección 15.1) lanza una excepción con la nota concreta que falló, no un genérico "dato inválido".

32. Lectura y escritura de ficheros

```
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) { System.out.println(linea); }
} catch (IOException e) {
```

```
        System.out.println("Error leyendo archivo");
    }
}
```

try-with-resources cierra el recurso solo, aunque haya excepción.

32.1 Escribir y leer varios recursos a la vez

```
try (BufferedReader br = new BufferedReader(new FileReader("entrada.txt"));
    BufferedWriter bw = new BufferedWriter(new FileWriter("salida.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        bw.write(linea.toUpperCase());
        bw.newLine();
    }
} catch (IOException e) {
    System.out.println("Error de E/S: " + e.getMessage());
}
```

try-with-resources con varios recursos los cierra **en orden inverso** al que se abrieron, automáticamente, incluso si algo falla a mitad. Antes de Java 7 esto era un **finally** anidado y propenso a errores.

32.2 Leer un CSV línea a línea y convertirlo en objetos

```
public static List<Alumno> leerAlumnosDesdeCSV(String ruta) {
    List<Alumno> alumnos = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
        String linea;
        while ((linea = br.readLine()) != null) {
            String[] partes = linea.split(","); // "Ana,8.5" → ["Ana", "8.5"]
            alumnos.add(new Alumno(partes[0], Double.parseDouble(partes[1])));
        }
    } catch (IOException e) {
        throw new RuntimeException("No se pudo leer el fichero", e);
    }
    return alumnos;
}
```

Este patrón — leer texto plano línea a línea, partir por un separador y construir objetos — es exactamente cómo funcionan por dentro los "importadores" de datos sencillos en aplicaciones reales, antes de dar el salto a formatos más estructurados (JSON, o directamente una base de datos vía JDBC, sección 35).

`Double.parseDouble()` lanza `NumberFormatException` (sección 31) si el texto no es un número válido — con datos que vienen de fuera (un fichero, un usuario), nunca asumas que el formato va a ser siempre correcto.

33. Principios de diseño: SRP y programar contra interfaces

✗ Mal: mezcla responsabilidades

```
class Usuario {
    void guardarEnArchivo() {}
    void imprimir() {}
}
```

✓ Bien: una responsabilidad por clase

```
class Usuario {}
class UsuarioService { void guardar() {} }
```

Y como en la sección 13: colecciones y dependencias con el tipo interfaz, no la clase concreta.

33.1 La Ley de Deméter: "no hables con extraños"

Regla informal muy útil: un método solo debería llamar a métodos de (1) sí mismo, (2) sus propios atributos, (3) sus parámetros, o (4) objetos que él mismo crea. No debería "atravesar" varios objetos encadenados para llegar a lo que necesita.

```
// ✗ Mal: encadena getters a través de tres objetos – conoce demasiado de la estructura interna
double ciudad = pedido.getCliente().getDireccion().getCiudad().length();

// ✓ Bien: cada clase expone lo que necesita, sin exponer CÓMO está montada por dentro
double ciudad = pedido.getCiudadDelCliente();
```

El problema del primer caso no es solo estético: si `Cliente` cambia cómo guarda su dirección, tienes que tocar código en todos los sitios que hacían ese encadenado, en vez de solo en un método de `Pedido`. Es la misma idea de fondo que "programar contra interfaces" (arriba): esconder los detalles de implementación para que un cambio interno no se propague a todo el proyecto.

34. Arquitectura en capas: Model / Service / Controller

```
class UsuarioController {
    private UsuarioService service = new UsuarioService();
    public void ejecutar() { service.guardar(); }
}
```

34.1 Las tres capas juntas, con un ejemplo completo

```
// Model: solo datos, sin lógica de negocio ni de persistencia
public record Alumno(int id, String nombre, double nota) {}

// Service: la lógica – reglas de negocio y acceso a datos (JDBC, sección 35)
public class AlumnoService {
    public List<Alumno> obtenerAprobados(List<Alumno> todos) {
        return todos.stream().filter(a -> a.nota() >= 5).toList(); // Predicate, sección 21
    }
}

// Controller: coordina – recibe la petición, llama al Service, decide qué devolver
public class AlumnoController {
    private final AlumnoService service;
    public AlumnoController(AlumnoService service) { this.service = service; } // inyectado, no creado dentro

    public void mostrarAprobados(List<Alumno> todos) {
        service.obtenerAprobados(todos).forEach(a -> System.out.println(a.nombre())); // Consumer, sección 22
    }
}
```

Fíjate en que el `Controller` recibe su `AlumnoService` ya creado por fuera (por el constructor), en vez de escribir `new AlumnoService()` dentro. Esto se llama **inyección de dependencias**: hace que puedas sustituir el `Service` real por uno de prueba en un test (sección 36) sin tocar el `Controller` para nada.

34.2 Por qué esto importa para el TFG

Cuando el proyecto crece — y un TFG con parte de front, parte de back y persistencia, como el que vas a hacer este año, crece rápido — esta separación es lo que evita que un cambio en una pantalla te obligue a tocar la lógica de negocio, o que cambiar de SQLite a otra base de datos te obligue a reescribir medio proyecto. Cada capa tiene un trabajo, y solo ese: el `Model` son datos, el `Service` decide "qué hacer", el `Controller` decide "cuándo llamarlo". Es la sección 33 (SRP) aplicada a la arquitectura completa del proyecto, no solo a una clase suelta.

Parte 6 · Bases de datos, testing y buenas prácticas

35. Conexión a bases de datos: JDBC y MongoDB

35.1 JDBC (SQL)

```
try (Connection con = DriverManager.getConnection(url, usuario, clave);
    PreparedStatement ps = con.prepareStatement("SELECT nombre FROM alumno WHERE edad > ?")) {
    ps.setInt(1, 18);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) { System.out.println(rs.getString("nombre")); }
    }
} catch (SQLException e) { e.printStackTrace(); }
```

35.1.1 JDBC dentro de la capa Service (arquitectura real)

En un proyecto real, el JDBC nunca va suelto en el `main`: vive dentro del `Service` (sección 34), que es quien conoce la base de datos. El `Controller` ni se entera de que por debajo hay SQL.

```
public class AlumnoService {
    private final String url = "jdbc:sqlite:alumnos.db";

    public void guardar(Alumno alumno) {
        String sql = "INSERT INTO alumno (nombre, nota) VALUES (?, ?)";
        try (Connection con = DriverManager.getConnection(url);
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, alumno.nombre());
            ps.setDouble(2, alumno.nota());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("Error guardando alumno", e); // checked → unchecked (sección 29.1)
        }
    }

    public List<Alumno> listarTodos() {
        List<Alumno> resultado = new ArrayList<>();
        String sql = "SELECT id, nombre, nota FROM alumno";
        try (Connection con = DriverManager.getConnection(url);
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                resultado.add(new Alumno(rs.getInt("id"), rs.getString("nombre"), rs.getDouble("nota")));
            }
        } catch (SQLException e) {
            throw new RuntimeException("Error leyendo alumnos", e);
        }
        return resultado;
    }
}
```

Fíjate: cada método abre y cierra su propia `Connection` con `try-with-resources` (sección 32) — nunca se comparte una conexión abierta entre llamadas sin control. Y las excepciones checked de JDBC (`SQLException`) se convierten aquí en unchecked, para no obligar a todas las capas superiores a declarar `throws SQLException` — exactamente el mismo patrón que ya vimos con las lambdas en la sección 29.1.

35.2 MongoDB (NoSQL)

```
MongoCollection<Document> productos = db.getCollection("productos");
productos.insertOne(new Document("nombre", "Ratón").append("precio", 19.99));
productos.find(Filters.gt("precio", 10));

// update: cambiar campos de los documentos que cumplan el filtro
productos.updateMany(Filters.eq("categoria", "periféricos"),
    Updates.set("enOferta", true));

// delete
productos.deleteOne(Filters.eq("nombre", "Ratón"));
```

A diferencia de SQL, aquí no hay tablas ni columnas fijas: cada `Document` es básicamente un mapa clave-valor (parecido a un JSON) y dos documentos de la misma colección ni siquiera tienen por qué compartir todos los campos. Es más flexible al empezar, pero esa misma flexibilidad significa que la validación de datos, que en SQL hace la propia base de datos, aquí recae más en tu código Java.

Misma idea de fondo en ambos: **CRUD** (Create, Read, Update, Delete). Cambia la sintaxis y el modelo de datos, no el concepto.

35.3 INSERT y UPDATE con JDBC

```
try (PreparedStatement ps = con.prepareStatement(
    "INSERT INTO alumno (nombre, nota) VALUES (?, ?)",
    Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "Jorge");
    ps.setDouble(2, 8.5);
    ps.executeUpdate();
    try (ResultSet claves = ps.getGeneratedKeys()) {
        if (claves.next()) { int idGenerado = claves.getInt(1); }
    }
}
```

Nunca concatenes texto para construir SQL ("SELECT * FROM x WHERE nombre = '" + nombre + "'"): es la puerta abierta a la **inyección SQL**. `PreparedStatement` con `?` existe justo para evitarlo — trátalo como obligatorio, no como buena práctica opcional.

35.4 Ejercicio integrador: excepciones + ficheros + JDBC

Une las secciones 31, 32 y 35: escribe un método `exportarACSV(List<Alumno> alumnos, String ruta)` que escriba una línea por alumno en un fichero, y si algún alumno tiene una nota fuera de 0-10, lance `NotaInvalidaException` (sección 31.2) **antes** de escribir nada — ni un fichero a medias ni una fila corrupta.

```
public void exportarACSV(List<Alumno> alumnos, String ruta) throws NotaInvalidaException {
    for (Alumno a : alumnos) { // validar TODO antes de tocar el fichero
        if (a.nota() < 0 || a.nota() > 10) {
            throw new NotaInvalidaException(a.nota());
        }
    }
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta))) {
        for (Alumno a : alumnos) {
            bw.write(a.nombre() + "," + a.nota());
            bw.newLine();
        }
    } catch (IOException e) {
        throw new RuntimeException("No se pudo exportar", e);
    }
}
```

La idea de validar todo **antes** de empezar a escribir (en vez de fallar a mitad) es un patrón que se repite mucho en código real: evita dejar el sistema en un estado a medias, que suele ser mucho más difícil de depurar que un fallo limpio al principio.

36. Testing básico con JUnit

```
@Test
void areaDeUnCirculoDeRadio2EsCorrecta() {
    Circulo circulo = new Circulo(2); // Arrange
    double area = circulo.area(); // Act
    assertEquals(12.566, area, 0.01); // Assert
}

@Test
void unaCuentaNoPuedeQuedarEnNegativo() {
    CuentaBancaria cuenta = new CuentaBancaria();
    cuenta.ingresar(50);
    assertThrows(IllegalStateException.class, () -> cuenta.retirar(100));
}
```

Método	Comprueba que...
<code>assertEquals(esperado, real)</code>	Son iguales (usa <code>equals()</code> , sección 12)
<code>assertTrue</code> / <code>assertFalse</code>	La condición es verdadera/falsa
<code>assertThrows(Excepcion.class, () -> ...)</code>	Se lanza esa excepción

Un test se re-ejecuta solo cada vez que tocas algo, y te avisa si rompiste algo que antes funcionaba. En un TFG, es la diferencia entre confiar a ciegas y saber con certeza.

36.1 Ciclo de vida de un test: `@BeforeEach` / `@AfterEach`

```
CuentaBancaria cuenta;

@BeforeEach // se ejecuta ANTES de cada @Test, deja todo limpio
void preparar() {
    cuenta = new CuentaBancaria();
    cuenta.ingresar(100);
}

@AfterEach // se ejecuta DESPUÉS de cada @Test, aunque falle
```

```

void limpiar() {
    System.out.println("Test terminado");
}

@Test
void retirarMenosDelSaldoFunciona() {
    cuenta.retirar(30);
    assertEquals(70, cuenta.getSaldo());
}

```

Con esto cada `@Test` parte de un estado idéntico y conocido — nada de un test que pasa o falla según en qué orden se ejecuten los demás, que es uno de los errores de testing más comunes al empezar.

36.2 Tests parametrizados: un test, varios casos

```

@ParameterizedTest
@ValueSource(ints = {2, 4, 6, 100})
void numerosPares(int numero) {
    assertTrue(numero % 2 == 0);
}

@ParameterizedTest
@CsvSource({"2, 4, 6", "3, 5, 8"})
void sumaDeDosNumeros(int a, int b, int esperado) {
    assertEquals(esperado, a + b);
}

```

Sin esto, probar la misma lógica con cinco entradas distintas serían cinco métodos `@Test` casi idénticos, copiados y pegados. Con `@ParameterizedTest`, un solo método se ejecuta una vez por cada fila de datos — y JUnit te dice exactamente cuál de las filas falló, no solo "algo falló".

36.3 La idea de los *mocks* (sin entrar en la librería todavía)

¿Cómo testear un `Controller` (sección 34) sin usar una base de datos real cada vez? Un **mock** es un objeto falso que sustituye a una dependencia real (el `Service`, por ejemplo) y devuelve respuestas predefinidas, para que el test se centre solo en la clase que realmente estás probando.

```

// Un "mock" hecho a mano, sin librerías — la idea central antes de automatizarla:
class AlumnoServiceFalso extends AlumnoService {
    @Override
    public List<Alumno> obtenerAprobados(List<Alumno> todos) {
        return List.of(new Alumno(1, "Falso", 10)); // respuesta fija, no toca ninguna base de datos
    }
}

AlumnoController controller = new AlumnoController(new AlumnoServiceFalso()); // inyectas el falso

```

Esto solo funciona limpio porque el `Controller` depende del **tipo interfaz/clase base** `AlumnoService`, no de una implementación concreta — la misma idea de programar contra interfaces de la sección 13 y 33, aplicada esta vez a hacer el código testeable. Librerías como **Mockito** automatizan justo esto (crean el mock por ti, sin que tengas que escribir una subclase a mano), pero la idea de fondo es exactamente la de arriba.

37. Debugging: cómo pensar como programador

✗ "¿Por qué no funciona?"

✓ "¿En qué línea deja de hacer lo que espero?"

Breakpoints, ejecución paso a paso, ventana de variables. Mejor que llenar el código de `System.out.println()`.

37.1 Ejemplo de cómo razonar un bug con breakpoints

Supón que un método que calcula la media de notas de una clase devuelve siempre 0. En vez de mirar todo el método a la vez:

1. Pon un breakpoint justo al entrar al método. Comprueba: ¿la lista de alumnos que llega tiene elementos? Si ya está vacía, el problema está antes, en quien la rellena.
2. Si tiene elementos, avanza línea a línea (*step over*) hasta el bucle o el stream. Mira el valor acumulado tras la primera vuelta. ¿Ya es 0 ahí? El problema está en cómo lees la nota de cada alumno (¿estás leyendo el campo correcto?).
3. Si el acumulado va bien pero el resultado final es 0, sospecha de una división entera: `total / alumnos.size()` con ambos `int` trunca decimales, y si el `total` es menor que `size()` incluso puede dar 0 directamente.

Ese proceso — acotar en qué línea concreta el valor real deja de coincidir con el esperado — es el 90% del debugging real, mucho más que "leer el código a ver si salta algo a la vista".

37.2 Errores de compilación más comunes al empezar

Mensaje	Causa típica
<code>cannot find symbol</code>	Variable/método mal escrito, o fuera de su ámbito (<code>{ }</code>).
<code>incompatible types</code>	Intentar meter un tipo donde se espera otro sin convertir (por ejemplo, un <code>double</code> en un <code>int</code> sin cast).

<code>missing return statement</code>	Un método con tipo de retorno declarado que tiene un camino (un <code>if</code> sin <code>else</code>) que no devuelve nada.
<code>variable might not have been initialized</code>	Usar una variable local antes de asignarle un valor en todos los caminos posibles.
<code>local variables referenced from a lambda must be final or effectively final</code>	Ver sección 19.1 — reasignaste una variable capturada por una lambda.
<code>unreported exception ...; must be caught or declared to be thrown</code>	Llamar a un método que lanza una excepción <i>checked</i> (sección 31) sin <code>try/catch</code> ni <code>throws</code> .

38. JavaDoc y buenas prácticas generales

```
/**
 * Calcula la media de las notas de una lista de alumnos.
 *
 * @param alumnos lista de alumnos a evaluar; no puede ser {@code null}
 * @return la media aritmética de las notas, o 0.0 si la lista está vacía
 * @throws IllegalArgumentException si algún alumno tiene una nota fuera de 0-10
 */
public static double calcularMedia(List<Alumno> alumnos) { /* ... */ }
```

IntelliJ genera el esqueleto solo (escribe `/**` encima del método y pulsa Enter), y luego se muestra automáticamente como tooltip al pasar el ratón por encima de una llamada al método en cualquier otro sitio del proyecto — no hace falta ir a buscar el código fuente para saber qué hace algo.

- Atributos siempre `private`.
- Colecciones y dependencias con el tipo interfaz.
- Una clase, una responsabilidad.
- `try-with-resources` para recursos que haya que cerrar.
- `equals()` / `hashCode()` / `toString()` juntos, o un `record`.
- Lambdas cortas y claras; si crece, un método con nombre.
- Nombres que se explican solos: `calcularMedia()` mejor que `calc()` o `metodo1()`.
- Commits pequeños y con mensaje claro — para el GitHub que quieres cuidar este año, esto se nota tanto como el propio código.

39. Resumen y plan de repaso

Bloque	Ya lo dominas si...
Fundamentos (1-8)	Escribes variables, bucles, arrays y métodos sin mirar apuntes
POO (9-15)	Explicas los 4 pilares y por qué <code>equals()</code> / <code>hashCode()</code> van juntos
Genéricos y colecciones (16-17)	Sabes por qué <code>List<String></code> es mejor que un <code>List</code> sin tipo
Interfaces funcionales (18-30)	Combinas <code>Predicate</code> / <code>Function</code> , distingues los 4 tipos de referencias a métodos y montas un pipeline de streams sin mirar la chuleta
Robustez (31-34)	Controlas excepciones, ficheros y separas Model/Service/Controller
Bases de datos y testing (35-36)	Distingues JDBC de MongoDB y escribes un test JUnit con Arrange-Act-Assert

Kotlin (este año) corre sobre la misma JVM y es muy parecido en conceptos — este repaso te sirve de base directa. Unity va en C#, no en Java.

39.1 Ideas de mini-proyectos para practicar por tu cuenta

Antes de meterte con el TFG o con Unity, tres ideas pequeñas (1-3 tardes cada una) que tocan casi todo lo de este documento y quedan bien en un GitHub organizado:

- **Gestor de biblioteca en consola:** clases `Libro`, `Usuario`, enum `EstadoLibro`, interfaz `Prestable`, y filtros con `Predicate` / streams para buscar libros disponibles por autor o categoría. Guarda los datos en un fichero de texto (sección 32) al cerrar el programa.
- **Analizador de notas con JDBC:** una tabla `alumno` en SQLite, capa Model/Service/Controller (sección 34), y un menú que use `Comparator` para ordenar por distintos criterios y `Collectors.groupingBy` para sacar estadísticas por clase. Con tests JUnit para el `Service`, sin tocar la base de datos real en los tests (como en el proyecto de ejemplo 3 de tu carpeta `Acceso a Datos`).
- **Motor de reglas con interfaces funcionales:** una lista de `Predicate<Pedido>` con nombre (`esUrgente`, `superaImporte`, `esClienteVip` ...) que combinas dinámicamente para decidir qué pedidos procesar primero. Es un ejercicio pequeño pero es exactamente el patrón que usan los motores de reglas de negocio reales.

39.2 Glosario rápido

Término	Qué es
JVM	Máquina virtual que ejecuta el bytecode Java; es lo que hace posible "write once, run anywhere".
Bytecode	Código intermedio (<code>.class</code>) que genera <code>javac</code> a partir de tu <code>.java</code> .

Garbage collector	Proceso automático de la JVM que libera memoria de objetos que ya nadie referencia.
Autoboxing / unboxing	Conversión automática entre un primitivo (<code>int</code>) y su clase envoltorio (<code>Integer</code>).
Encapsulación	Ocultar el estado interno de un objeto (atributos <code>private</code>) y exponer solo lo necesario.
Herencia	Una clase (<code>extends</code>) reutiliza y extiende el comportamiento de otra.
Polimorfismo	Un mismo método se comporta distinto según el tipo real del objeto en tiempo de ejecución.
Abstracción	Definir "qué se puede hacer" sin fijar "cómo", vía clases abstractas o interfaces.
Contrato de <code>equals()</code>	Reglas que debe cumplir toda implementación: reflexiva, simétrica, transitiva, consistente.
Type erasure	La información de los genéricos desaparece al compilar; en ejecución no existe <code>List<String></code> , solo <code>List</code> .
SAM	<i>Single Abstract Method</i> : la regla que define una interfaz funcional (un único método sin cuerpo).
Closure	Una lambda que "captura" variables de su entorno (deben ser effectively final).
Effectively final	Variable que nunca se reasigna tras su primera asignación, aunque no lleve <code>final</code> escrito.
Method reference	Sintaxis <code>Clase::metodo</code> para pasar un método ya existente como si fuera una lambda.
Stream	Secuencia de datos sobre la que se encadenan operaciones (<code>filter</code> , <code>map</code> ...), sin modificar la colección original.
Lazy evaluation	Retrasar un cálculo hasta que realmente se necesita (p. ej. <code>Supplier</code> en <code>orElseGet</code>).
Checked / unchecked	Excepciones que el compilador obliga a gestionar (<code>Exception</code>) o no (<code>RuntimeException</code>).
try-with-resources	Bloque <code>try(...)</code> que cierra automáticamente los recursos abiertos, incluso si hay excepción.
SRP	<i>Single Responsibility Principle</i> : cada clase debería tener un único motivo para cambiar.
PreparedStatement	Sentencia SQL parametrizada con <code>?</code> , segura frente a inyección SQL.
CRUD	Create, Read, Update, Delete — las cuatro operaciones básicas sobre datos persistentes.
Arrange-Act-Assert	Estructura estándar de un test: preparar datos, ejecutar la acción, comprobar el resultado.

39.3 Atajos de IntelliJ que vale la pena aprender ya

Atajo (Windows)	Qué hace
<code>Alt + Insert</code>	Generate: constructor, getters/setters, <code>equals()</code> / <code>hashCode()</code> / <code>toString()</code> ...
<code>Ctrl + Alt + L</code>	Reformatear el código (indentación, espacios) automáticamente.
<code>Shift + F6</code>	Renombrar una variable/método/clase en TODO el proyecto de forma segura.
<code>Ctrl + B</code> (o <code>Ctrl + Click</code>)	Ir a la definición de lo que tienes bajo el cursor.
<code>Alt + Enter</code>	Sugerencias rápidas del IDE (importar una clase, generar código que falta...).
<code>Shift + F10</code>	Ejecutar el programa.
<code>Shift + F9</code>	Ejecutar en modo debug (con los breakpoints activos, sección 37).
<code>F8</code> / <code>F7</code>	En debug: avanzar una línea (<i>step over</i>) / entrar dentro del método (<i>step into</i>).
<code>Ctrl + /</code>	Comentar/descomentar la línea o selección actual.

No hace falta memorizarlos todos de golpe — ve incorporando uno o dos por semana. `Alt+Insert` y `Shift+F6` son, con diferencia, los que más tiempo ahorran desde el primer día.

39.4 Proyecto integrador comentado: gestor de tareas

Para cerrar, un ejemplo pequeño pero completo que junta records, excepciones propias, ficheros y varias interfaces funcionales de la Parte 4 — el tipo de programa que puedes ampliar tú mismo como mini-proyecto de la sección 39.1.

```
public record Tarea(String titulo, int prioridad, boolean completada) {
    public Tarea completar() { return new Tarea(titulo, prioridad, true); } // immutable: devuelve una copia
}

public class TareaNoEncontradaException extends RuntimeException {
    public TareaNoEncontradaException(String titulo) { super("No existe la tarea: " + titulo); }
}

public class GestorTareas {
    private List<Tarea> tareas = new ArrayList<>();

    public void añadir(String titulo, int prioridad) {
        tareas.add(new Tarea(titulo, prioridad, false));
    }

    public void completar(String titulo) {
        Predicate<Tarea> esEsta = t -> t.titulo().equals(titulo);
        Tarea tarea = tareas.stream()
            .filter(esEsta)
```

```

        .findFirst()
        .orElseThrow(() -> new TareaNoEncontradaException(titulo)); // Supplier de la excepción: solo se crea si hace
falta

        tareas.remove(tarea);
        tareas.add(tarea.completar());
    }

    public List<Tarea> pendientesPorPrioridad() {
        return tareas.stream()
            .filter(t -> !t.completada())
            .sorted(Comparator.comparingInt(Tarea::prioridad).reversed())
            .toList();
    }

    public void guardarEnFichero(String ruta) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(ruta))) {
            for (Tarea t : tareas) {
                bw.write(t.titulo() + ";" + t.prioridad() + ";" + t.completada());
                bw.newLine();
            }
        } catch (IOException e) {
            throw new RuntimeException("No se pudo guardar: " + e.getMessage(), e);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        GestorTareas gestor = new GestorTareas();
        gestor.añadir("Repasar interfaces funcionales", 3);
        gestor.añadir("Montar proyecto JDBC", 2);
        gestor.añadir("Leer sobre Kotlin", 1);

        gestor.completar("Leer sobre Kotlin");

        gestor.pendientesPorPrioridad()
            .forEach(t -> System.out.printf("[P%d] %s\n", t.prioridad(), t.titulo()));

        gestor.guardarEnFichero("tareas.txt");
    }
}

```

En menos de 50 líneas hay: un `record` inmutable con un método propio (sección 15), una excepción personalizada con contexto (sección 31.2), un `Predicate` con nombre y una referencia a método dentro de `Comparator.comparingInt().reversed()` (secciones 21 y 26), un `Supplier` pasado a `orElseThrow()` que solo construye la excepción si realmente hace falta (sección 22.2), y `try-with-resources` para el fichero (sección 32). Si entiendes por qué está cada pieza donde está, tienes todo este documento asentado de verdad — no solo leído.

39.5 Autoevaluación: 15 preguntas para comprobar que lo tienes

Sin mirar el documento, respóndelas mentalmente (o en voz alta). Si alguna se te escapa, ya sabes a qué sección volver — están en orden.

1. ¿Qué diferencia hay entre `==` y `equals()` con objetos?
2. ¿Por qué `private static int total` se comparte entre todos los objetos de la clase?
3. Nombra los 4 pilares de la POO y una frase de cada uno.
4. Si sobrescribes `equals()`, ¿qué otro método tienes que sobrescribir también, y por qué?
5. ¿Cuándo usarías una clase abstracta y cuándo una interfaz?
6. ¿Qué te da gratis un `record` que no te da una clase normal?
7. ¿Qué problema resuelven los genéricos que no resolvía usar `Object` a secas?
8. ¿Qué es una interfaz funcional, en una frase?
9. ¿Qué significa que una variable capturada por una lambda tenga que ser "effectively final"?
10. Diferencia entre `andThen()` y `compose()` en `Function`.
11. ¿Por qué `orElseGet()` es mejor que `orElse()` cuando calcular el valor por defecto es costoso?
12. Nombra los 4 tipos de referencia a método.
13. ¿Qué hace `try-with-resources` que no hacía un `try/finally` normal?
14. ¿Por qué nunca debes concatenar texto para construir una consulta SQL?
15. ¿Qué estructura sigue un test JUnit típico (las tres letras)?

Si has llegado hasta aquí y puedes responder la mayoría sin mirar, el repaso ha cumplido su función — el resto es programar, que es donde esto se queda de verdad grabado.

Con esto tienes todo lo que traías más lo que fuiste pidiendo por el camino, y sobre todo las interfaces funcionales explicadas de verdad y no solo por encima. Ahora toca practicar: coge cada bloque y haz un ejercicio pequeño antes de pasar al siguiente.